

claude-windows / fa4dc4eb-4705-47fd-b044-4f017b586b15

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-inde  
xer\fa4dc4eb-4705-47fd-b044-4f017b586b15\subagents\workflows\wf_ac0cbb29-6bf\agent-  
a3efe699d7be1c214.jsonl`  
- SHA-256: `0511abe74307b263bdf503a43bcc2e83582ecac533af1262a748a3dcf251faf`  
- Source modified: `2026-06-19T15:52:56+00:00`  
- Imported at: `2026-07-05T16:48:22+00:00`  
- project: `wf_ac0cbb29-6bf`  
- session_id: `fa4dc4eb-4705-47fd-b044-4f017b586b15`
```

Transcript

1. user (2026-06-19T15:52:06.178Z)

You are auditing a badminton rally-detection codebase for DOC/COMMENT STALENESS ? statements that CONTRADICT the current code or data.

HARD RULES:

- Report ONLY factual staleness with CODE EVIDENCE (file:line or symbol) that proves it. Open the cited code and confirm before reporting.
- NEVER invent a fact or a status. If a thing is built-but-default-OFF / unwired / unmeasured, say EXACTLY that ? do not overclaim "in production".
- A "[design]" tag is stale ONLY if the thing is actually BUILT in code (cite the symbol).
- DO NOT propose subjective rewrites, style/tone changes, or improvements. DO NOT relitigate owner-gated DECISIONS (owned-model roadmap, licensing/MIT-Apache stance, product/strategy choices, genuinely-still-design proposals).
- Preserve the document's voice and formatting; make the SMALLEST surgical edit. stale_text MUST be copied verbatim with enough context to be unique.
- IGNORE docs/archives/** (terminal snapshots) and any memory/ file.

Working dir is the repo root; use Read/Grep/Glob freely.

ADVERSARIALLY VERIFY these proposed staleness fixes for ONE file: docs/R2_EVAL_METRIC_DESIGN.md. For EACH proposed fix: (1) open docs/R2_EVAL_METRIC_DESIGN.md and confirm stale_text appears VERBATIM (tighten/expand it to an exact, UNIQUE match if needed ? this text will be applied by exact string match, so it must be perfect); (2) re-open the cited code yourself and confirm the claim is genuinely contradicted by the CURRENT code; (3) confirm corrected_text is accurate, preserves voice, and is minimal; (4) DROP it if it is subjective, an owner-gated decision, already-correct, or you cannot verify the evidence. Output ONLY the survivors (verdict confirm or revise), with stale_text guaranteed to be an exact unique substring of the file.

PROPOSED FIXES:

```
[  
  {  
    "file": "docs/R2_EVAL_METRIC_DESIGN.md",  
    "stale_text": "- **Wire into `backend/eval/rally_seg_eval.py` ? emit the R2 block  
**alongside** tIoU (don't remove tIoU);\n  add `--tau-start/--tau-end/--tau-sweep`. Surface in  
the experiment leaderboard.",  
    "corrected_text": "- **Wire into `backend/eval/rally_seg_eval.py` ? emit the R2 block  
**alongside** tIoU (don't remove tIoU);\n  add `--tau-start/--tau-end` (both landed; the ?-sweep  
helper `tolerance_metrics.tolerance_sweep` exists\n  but is not yet surfaced as a  
`rally_seg_eval` CLI flag). Surface in the experiment leaderboard.",  
    "why_stale": "Claims a `--tau-sweep` CLI flag was wired into rally_seg_eval.  
backend/eval/rally_seg_eval.py main() defines only `--tau-start` (:467) and `--tau-end` (:469);  
there is no `--tau-sweep` argument and rally_seg_eval never calls  
`tolerance_metrics.tolerance_sweep` (grep for tolerance_sweep/sweep in rally_seg_eval.py: no  
matches). The doc's own status header says IMPLEMENTED, but this named flag is not.",  
    "severity": "low"  
  },  
  {
```

```

    "file": "docs/R2_EVAL_METRIC_DESIGN.md",
    "stale_text": "- `tolerance_metrics.py` + tests merged (default-additive); `rally_seg_eval`
reports the R2 decomposition +\n ?-sweep next to tIoU; the corpus (n?8) re-scored under R2 and
recorded in QUALITY_ITERATIONS.",
    "corrected_text": "- `tolerance_metrics.py` + tests merged (default-additive);
`rally_seg_eval` reports the R2 decomposition\n next to tIoU (the ?-sweep helper exists in
`tolerance_metrics` but is not yet wired into the\n `rally_seg_eval` report); the corpus (n?8)
re-scored under R2 and recorded in QUALITY_ITERATIONS.",
    "why_stale": "Definition-of-done asserts `rally_seg_eval` reports the ?-sweep next to tIoU.
The decomposition (P/R/F1@? + boundary error + over-seg) is reported by
rally_seg_eval.format_report (:349-363), but the ?-sweep is not: rally_seg_eval.py never
imports/calls tolerance_metrics.tolerance_sweep (:274 of tolerance_metrics.py) nor renders a
sweep table.",
    "severity": "low"
  },
  {
    "file": "docs/R2_EVAL_METRIC_DESIGN.md",
    "stale_text": "- `boundary_error_report(matches) -> {?start, ?end: signed/abs
median+IQR}`.",
    "corrected_text": "- `boundary_error_report(preds, gts) -> {?start, ?end: signed/abs
median+IQR}` (computed over\n unconditional best-overlap matches internally ? see the
refinement note above).",
    "why_stale": "Wrong function signature: the implemented `boundary_error_report` takes
`(preds, gts)`, not `(matches)` (backend/eval/tolerance_metrics.py:122 `def
boundary_error_report(preds, gts)`). It computes best-overlap matches internally ? consistent
with this doc's own $0 refinement note ? so the `(matches)` argument shown here contradicts the
shipped code.",
    "severity": "low"
  }
]

```

2. assistant (2026-06-19T15:52:08.530Z)

I'll verify each proposed fix against the actual file and code. Let me start by reading the target document and the cited code files.

3. user (2026-06-19T15:52:09.483Z)

```

1      # R2 ? Task-faithful rally-boundary eval metric (tolerance-match + over-seg guard)
2
3      **Status:** **IMPLEMENTED (augmenting), 2026-06-17** ? owner-reviewed + decisions locked
2026-06-17;
4      `backend/eval/tolerance_metrics.py` + `rally_seg_eval` reporting landed (default-
additive; tIoU stays the
5      gate pending the ablation experiment). Gates the rally-end work (R4) and the dual-eval-
set regression gate.
6      A focused increment of the rally-quality project
([RALLY_QUALITY_RESEARCH.md](RALLY_QUALITY_RESEARCH.md) §7;
7      the day-1 validation that motivates it:
[REAL_FOOTAGE_VALIDATION_REPORT.md](REAL_FOOTAGE_VALIDATION_REPORT.md);
8      the directive: `eval-boundary-tolerance` + `eval-dual-set-regression`).
9
10     > **Implementation refinement (2026-06-17):** §2.3.2's boundary-error distribution is
computed over
11     > **unconditional best-overlap** matches, NOT the within-? 1:1 matches ? a ?-gated set
is survivorship-biased
12     > (loose ends fail the gate and vanish, hiding the very end-deficit the diagnostic
exists to surface; that
13     > within-? deficit instead shows up as low `tol_recall`). Matches the day-1 validation
method. Measured on the
14     > n=9 corpus: **|?start| 1.25 s vs |?end| 2.19 s** ? "start?solved, end=the gap",
confirmed.
15
16     ---
17

```

18 ## 0. TL;DR

19 `tIoU-F1@0.5` is the wrong ruler for rally detection, in **both** directions: it over-penalizes the

20 **intrinsically fuzzy rally START** (the ~1?1.5 s service window), and its overlap matching is murky about

21 **over-production**. R2 replaces the headline with a **tolerance-match** metric under strict 1:1 (Hungarian)

22 assignment, reported as a **decomposition** ? `P@? / R@? / F1@?` + **boundary-error** distributions split

23 into start/end + an **over-segmentation guard** + a **?-sweep** ? not a single number.

24

25 **Honest**, slightly uncomfortable consequence (measured, see §3): under R2 the local detector's headline

26 number *drops* and the gap to Gemini *widens* vs what tIoU showed ? because R2 faithfully charges for the

27 1.5× over-production and the loose ends that tIoU's overlap matching partly absorbed. That is the point:

28 R2 splits "behind Gemini" into its two real causes ? **(a) over-production [precision]** and **(b) loose**

29 rally-ends [the R4 lever] ? and forgives only the inherent start fuzziness. It is a more honest ruler,

30 not a flattering one.

31

32 ---

33

34 ## 1. Why `tIoU@0.5` is the wrong ruler (grounded in the n=2 ground-truth clips)

35 1. **Over-penalizes the START**. For a 5.7 s rally, `tIoU ? 0.5` tolerates only ~±1.9 s of boundary shift

36 (`? = D·(1?)/(1+?)`). The service window (stance ? racquet contact) alone is ~1?1.5 s ? even **Gemini's**

37 median $|?start|$ is 0.98?1.46 s. So tIoU spends most of its budget on a boundary that is *intrinsically*

38 undefined to ~1 s. (Owner directive: don't strictly align starts; tolerate ~±2 s.)

39 2. **Count parity is a deceptive proxy**. mahadevpura-2 hit 1.03× of Gemini's count yet $F1@0.5 = 0.150$?

40 right count, wrong boundaries. A count- or overlap-based view hides this.

41 3. **It flips config rankings on noise**. The hard-cap was golden-neutral at n=6 (+0.006 n.s.) but "best" at

42 n=7 once a continuously-active clip entered ? a metric artifact at the 0.5 cliff, not a quality change.

43 4. **The deficits we must see are start?end-symmetric**. Local's $|?start|$? Gemini's; the **entire** gap is

44 the rally-**END** ($|?end|$ 2.1?3.1 s vs Gemini ~0.8?1.2 s). A single blended IoU cannot show that ? and

45 directing R4 needs exactly that decomposition.

46

47 ---

48

49 ## 2. The metric

50

51 ### 2.1 Match rule (asymmetric tolerance ? forgive the start, expose the end)

52 A predicted window matches a golden rally iff **``|?start| ? ?_start` AND `|?end| ? ?_end`**.

53 - **``?_start` generous (~2.0 s)** ? absorbs the service-window/annotator start fuzziness (the part that is

54 *not* a model deficit).

55 - **``?_end` tighter (~1.5 s, but IAA-calibrated ? see §2.4)** ? the rally-end IS a sharp physical event

56 (shuttle down/net/out), so a loose end *is* a real deficit we want the metric to expose, not forgive.

57

58 > A *symmetric* ±2 s tolerance is tempting (the owner's stated bar) but the preview (§3) shows it is **not**

59 > automatically more lenient than tIoU 0.5 and it blurs exactly the end-deficit we need to see. Asymmetry is

```

60     > the point: relax the dimension that's intrinsically fuzzy (start), keep honest the one
        that's a real event (end).
61
62     ### 2.2 Assignment ? strict 1:1 (Hungarian), never overlap/greedy
63     Match references to predictions by **global Hungarian assignment** (max total matches
        under the tolerance
64     gate), one-to-one. This is what makes **over-production cost precision directly**: `P@?
        = matched / n_pred`
65     crashes when the detector emits 75 windows for 52 rallies. (tIoU's overlap matching let
        extras "match
66     something"; the W1 over-segmentation pendulum was barely costed.)
67
68     ### 2.3 Report the DECOMPOSITION, never one number
69     1. **`P@?`, `R@?`, `F1@?`** ? precision (over-production), recall (did we find the rally
        within tolerance), F1.
70     2. **Boundary-error distributions** ? median **+ IQR** of signed **and** absolute
        `?start`, `?end` over matched
71     pairs, **split**. This is the metric that says **"start solved, end open" and aims
        R4.
72     3. **Over-seg guard (mandatory):** `split_count` (preds covering one ref ? W1 over-seg)
        and `merge_count`
73     (refs under one pred ? the baseline mega-window). The **original** rule was strict per-
        video: **a promotion
74     may not increase either on ANY video. **REFINED to a NET guard, 2026-06-17 (R1,
        owner-approved) **? the
75     strict rule is correct for an **incremental** cue but **mis-fires on a structural mega-
        window?bounded change**
76     (splitting a mega-window necessarily lifts `split_count` from ~0, and the raw merge
        **count** is non-monotonic:
77     one window hiding 40 rallies is `count=1, rate=1.0`; bounding it into pieces that
        each glue 2 neighbours is
78     `count=9, rate=0.5` ? FEWER rallies hidden). The net guard scores over-seg on the
        **normalized rates** as one
79     weighted cost per video (`w_merge.merge_rate + w_split.split_rate`, merges weighted
        higher ? a merge hides a
80     rally = recall loss, a split only fragments one), and **passes iff (a) the corpus-
        mean net cost does not rise
81     AND (b) no video's `merge_rate` regresses. **Implemented as
        `tolerance_metrics.over_seg_guard` (it always
82     reports the strict verdict too; revert = read `strict_passes`). See
        QUALITY_ITERATIONS.md "R1" for the measured
83     verdict (milestone: mean `merge_rate` 0.694?0.150, net PASS, strict BLOCK).
84     4. **?-sweep** (`? ? {1.5, 2.0, 2.5, 3.0}`, symmetric for the trend; asymmetric pair for
        the headline) ?
85     because the convention/IAA noise is large, a single ? misleads (§3). Report the
        curve.
86     5. **`tIoU`/`mIoU` demoted to a SECONDARY trend-line diagnostic ** (keeps the paper-
        ablation backbone
87     comparable) ? **not the gate.**
88
89     ### 2.4 Choosing ? ? it MUST be calibrated to inter-annotator agreement
90     The preview (§3) shows even **Gemini** scores `tolF1@1.5 = 0.20` on mp2 ? i.e. on ~half
        the rallies the
91     golden END differs from Gemini's by >1.5 s. That is plausibly **label/convention noise,
        not model error**:
92     we'd be measuring how two humans (or human-vs-Gemini) disagree on "when did the rally
        end." **Therefore
93     `?_end` must be ? the inter-annotator end-agreement ** , which we do not yet know.
        **Action:** a small **IAA
94     study ** ? a 2nd labeler re-labels 2?3 already-golden videos; set `?_start`/`?_end` to
        ~the 90th-percentile
95     pairwise human |?start|/?end|. Until then **freeze provisional ?_start=2.0, ?_end=1.5
        and always report the
96     sweep ** , with the `? = D.(1??)/(1+?)` derivation recorded so the choice is auditable.
97

```

```

98      ---
99
100     ## 3. Preview on the two ground-truth clips (greedy-1:1 approximation; Hungarian in the
impl)
101
102     Symmetric-? sweep, F1@? (local-best = the per-clip best config: mp2 W1+W2+hardcap,
GX010128 W1+W2):
103
104     | clip / method | tolF1@1.5 | @2.0 | @2.5 | @3.0 | (tIoU-F1@0.5) |
105     |---|---|---|---|---|---|
106     | mahadevpura-2 / **Gemini** | 0.203 | 0.354 | 0.506 | **0.557** | 0.557 |
107     | mahadevpura-2 / local | 0.111 | 0.148 | 0.222 | 0.278 | 0.296 |
108     | GX010128 / **Gemini** | 0.615 | 0.769 | 0.813 | **0.857** | 0.813 |
109     | GX010128 / local | 0.189 | 0.315 | 0.346 | 0.362 | 0.457 |
110
111     **Reading the preview (all honest):**
112     - **`tolF1@~3.0 ? tIoU-F1@0.5`** ? the sweep is a smooth, interpretable knob that
*contains* tIoU as a point;
113     good (continuity with the frozen baseline).
114     - **The local?Gemini gap WIDENS under the tighter, 1:1 tolerance** (GX010128 ratio 0.56
at tIoU0.5 ?
115     **0.41** at ?2.0). R2 charges local for the 1.5x over-production (precision) and the
loose ends ? exactly
116     the deficits tIoU under-counted. **This is the metric working, not local regressing.**
117     - **Even Gemini drops at small ?** (mp2 0.557?0.203 from ?3.0?1.5) ? strong evidence the
**golden END
118     convention carries >1.5 s of noise** ? §2.4's IAA calibration is a prerequisite, not a
nicety.
119     - Boundary-error medians (the robust diagnostic, from the validation report): local
|?start| 0.78?1.74 s
120     (? Gemini 0.98?1.46) vs local |?end| 2.12?3.12 s (? Gemini 0.76?1.19) ? **the headline
R4 signal.**
121
122     ---
123
124     ## 4. Promotion / "trustworthiness" rule (small-n)
125     A candidate is promotable only if **all** hold:
126     1. **Paired per-video sign-test sweep on F1@? is near-unanimous: **6/0 @ n=6
(p=0.031)** or **7/0 @ n=7
127     (p=0.016)** (8/0 @ n=8, p=0.008); 5/1 or 6/1 is NOT significant. Run on the
**growing-golden** set.
128     2. **No over-seg regression** ? the **net** over-seg guard passes (§2.3.3): the corpus-
mean weighted
129     merge/split-rate cost does not rise AND no video's `merge_rate` regresses. (Was the
strict per-video
130     count rule; refined to net on 2026-06-17 ? see §2.3.3.)
131     3. **Effect size + bootstrap CI** reported as an honesty band (expect it to span 0 at
this n; use it to
132     separate "real" from "directionally-clean-but-negligible," not as the pass/fail).
133     4. **Stratify, don't blend** ? report the continuously-active blob clip (mahadevpura-1)
as its own stratum
134     (its near-degenerate scores can swing a small-n mean).
135     5. **No regression on EITHER eval set** (raw + growing-golden) ? the dual-set
discipline.
136
137     ---
138
139     ## 5. Implementation plan (pure, offline, additive ? zero production risk)
140     - **`backend/eval/tolerance_metrics.py`** (pure, GPU-free, unit-testable):
141     - `match_1to1(preds, gts, ?_start, ?_end) -> (matches, P, R, F1)` ? Hungarian via
142     `scipy.optimize.linear_sum_assignment` if available, else a pure O(n3) fallback
(corpus n is tiny).
143     - `boundary_error_report(preds, gts) -> {?start, ?end: signed/abs median+IQR}`
(computed over
144     unconditional best-overlap matches internally ? see the refinement note above).

```

```

145     - `over_seg_report(preds, gts) -> {split_count, merge_count}` (reuse the bipartite
logic from
146     `segmentation_metrics.merge_split_report`).
147     - **Wire into `backend/eval/rally_seg_eval.py`** ? emit the R2 block **alongside** tIoU
(don't remove tIoU);
148     add --tau-start/--tau-end` (both landed; the ?-sweep helper
`tolerance_metrics.tolerance_sweep` exists
149     but is not yet surfaced as a `rally_seg_eval` CLI flag). Surface in the experiment
leaderboard.
150     - **`run_regression.py`** ? add the dual-set R2 aggregate + the over-seg-guard gate.
*(Status: the guard
151     primitive landed as `tolerance_metrics.over_seg_guard` and is wired into
`rally_seg_eval.compare()`/`--vs`
152     ? the natural home, since `rally_seg_eval` loads the golden manifest + trajectories.
The dual-set
153     `run_regression.py` aggregate is still a separate future item.)*
154     - **Tests** ? synthetic intervals: exact-match, start-fuzzy-within-?, end-loose-
beyond-?, over-production
155     (precision crash), the Hungarian-vs-greedy tie case, empty inputs.
156     - **Default-additive ? ablation-gated replacement:** R2 is reported next to tIoU
(augment). The *gate*
157     switches to R2 only after a dedicated **ablation experiment** on the golden corpus
shows R2 ranks candidates
158     at least as sensibly as tIoU ? so the promotion bar never changes silently mid-flight.
When tIoU is retired
159     as the gate it is **deprecated with documentation, not deleted** (kept as the
secondary trend-line + the
160     `QUALITY_ITERATIONS.md` paper-ablation backbone, with a dated note recording the
ablation evidence for the
161     switch). Owner decision, 2026-06-17 (§7).
162
163     ---
164
165     ## 6. Definition of done
166     - `tolerance_metrics.py` + tests merged (default-additive); `rally_seg_eval` reports the
R2 decomposition
167     next to tIoU (the ?-sweep helper exists in `tolerance_metrics` but is not yet wired
into the
168     `rally_seg_eval` report); the corpus (n?8) re-scored under R2 and recorded in
QUALITY_ITERATIONS.
169     - The IAA study (§2.4) scoped (it can land after R2's code; R2 ships with provisional ?
+ the sweep).
170     - `run_regression.py` carries the dual-set R2 gate + the per-video over-seg guard ? but
R2 **augments** tIoU
171     until the ablation experiment (§7) validates it as the gate; only then does the gate
switch.
172     - tIoU/mIoU retained as a secondary trend-line; when retired as the gate, **deprecated-
with-docs (kept for
173     posterity)** per the §7 owner decision ? a dated note records why and links the
ablation evidence.
174
175     ---
176
177     ## 7. Decisions
178     **Locked (recommended, design-level):** Hungarian 1:1 (not greedy/overlap); report the
decomposition + sweep,
179     never one number; tIoU demoted to secondary; R2 default-additive until validated;
boundary-error split
180     start/end is the R4-aiming diagnostic.
181
182     **Locked (owner review, 2026-06-17):**
183     - **?_end asymmetric (1.5 s) vs ?_start (2.0 s) ? ADOPTED.** Asymmetry is the point:
forgive the intrinsically
184     fuzzy start, keep the rally-end honest. Your " $\pm 2$  s, not more" bar maps to
`?_start=2.0`; `?_end` stays the

```

```

185     tighter, IAA-calibrated knob ($2.4).
186     - **IAA study after R2 code ? ADOPTED.** R2 ships now with provisional `?_start=2.0 /
?_end=1.5` + the full
187     sweep; the IAA study lands next to fix ?. R2's code is not blocked on it.
188     - **Gate: AUGMENT now, then ablation-gate the eventual replacement, then deprecate-with-
docs ? ADOPTED.**
189     R2 is reported **alongside** tIoU (augment) until validated. tIoU is **not** swapped
out silently: before
190     R2 becomes the promotion gate, run a dedicated **ablation experiment** on the golden
corpus showing R2
191     ranks candidates at least as sensibly as tIoU (and exposes the deficits tIoU hid).
When tIoU is finally
192     retired as the gate, it is **formally deprecated with documentation ? kept for
posterity, never deleted**:
193     it remains a secondary trend-line and the paper-ablation backbone
(`QUALITY_ITERATIONS.md`), with a dated
194     note recording *why* it was superseded and the ablation evidence behind the switch.
(Consistent with the
195     repo's "retain weak/superseded signals; numbers inform, foundations decide" norm.)
196

```

4. user (2026-06-19T15:52:09.747Z)

```

1     ""R2 ? task-faithful rally-boundary eval (tolerance-match + over-segmentation guard).
2
3     The headline rally metric (tIoU-F1 is kept as a SECONDARY diagnostic): a prediction
matches a
4     golden rally iff ``|?start| ? ?_start AND |?end| ? ?_end`` under a **strict 1:1
assignment**, so
5     over-production costs precision directly. Reported as a DECOMPOSITION ? ``P/R/F1@?`` +
start/end
6     boundary-error distributions + an over-seg guard (split/merge counts) + a ?-sweep ?
never one
7     number (a single ? misleads; see the design doc).
8
9     Locked decisions + rationale: ``docs/R2_EVAL_METRIC_DESIGN.md`` (owner-reviewed
2026-06-17):
10    asymmetric ? (``?_start=2.0`` forgives the ~1?1.5 s service window; ``?_end=1.5`` keeps
the
11    rally-end honest ? IAA-calibrated later); augment tIoU now, ablation-gate any eventual
swap.
12
13    Pure + stdlib; **numpy/scipy are OPTIONAL** ? used for the exact Hungarian assignment
when present,
14    else a deterministic greedy 1:1 fallback. Both agree on the temporally-ordered, near-
diagonal
15    eligibility graphs that rally intervals produce, so scores are environment-independent.
16    ""
17    from __future__ import annotations
18
19    import statistics
20    from typing import Any, Dict, List, Optional, Tuple
21
22    Interval = Tuple[float, float]
23    #: (pred_idx, gt_idx, dstart_signed, dend_signed) ? signed = pred ? gt (positive = pred
is late).
24    Match = Tuple[int, int, float, float]
25
26    #: Provisional ? (the design's locked starting point; fixed by the IAA study later).
Always
27    #: report the sweep alongside, since the golden END convention carries >1.5 s of noise.
28    TAU_START: float = 2.0
29    TAU_END: float = 1.5
30    SWEEP_TAU: Tuple[float, ...] = (1.5, 2.0, 2.5, 3.0)
31

```

```

32
33     def _eligible(p: Interval, g: Interval, tau_start: float, tau_end: float) -> bool:
34         return abs(p[0] - g[0]) <= tau_start and abs(p[1] - g[1]) <= tau_end
35
36
37     def _greedy_match_1to1(preds: List[Interval], gts: List[Interval],
38         tau_start: float, tau_end: float) -> List[Match]:
39         """Deterministic greedy 1:1: eligible (pred, gt) pairs in ascending combined
boundary error,
40         assigned first-come and skipping used indices. Optimal-or-near on the near-diagonal
eligibility
41         graph rally intervals produce. The numpy/scipy-free path (and the CI path)."""
42         pairs: List[Tuple[float, float, int, int]] = []
43         for i, p in enumerate(preds):
44             for j, g in enumerate(gts):
45                 if _eligible(p, g, tau_start, tau_end):
46                     # sort key: combined error, then (i, j) for a fully deterministic
tiebreak
47                     pairs.append((abs(p[0] - g[0]) + abs(p[1] - g[1]), float(i * 1e-6 + j *
1e-9), i, j))
48         pairs.sort()
49         used_p: set = set()
50         used_g: set = set()
51         matches: List[Match] = []
52         for _, _, i, j in pairs:
53             if i in used_p or j in used_g:
54                 continue
55             used_p.add(i)
56             used_g.add(j)
57             matches.append((i, j, preds[i][0] - gts[j][0], preds[i][1] - gts[j][1]))
58         matches.sort(key=lambda mm: mm[1])
59         return matches
60
61
62     def _hungarian_match_1to1(preds: List[Interval], gts: List[Interval],
63         tau_start: float, tau_end: float) -> Optional[List[Match]]:
64         """Exact max-cardinality (then min total boundary error) via scipy. Returns ``None``
when
65         numpy/scipy are unavailable so the caller falls back to greedy."""
66         try:
67             import numpy as np
68             from scipy.optimize import linear_sum_assignment
69         except Exception:
70             return None
71         n, m = len(preds), len(gts)
72         size = max(n, m)
73         big = 1.0e6
74         cost = np.full((size, size), big, dtype=float)
75         for i, p in enumerate(preds):
76             for j, g in enumerate(gts):
77                 if _eligible(p, g, tau_start, tau_end):
78                     cost[i, j] = abs(p[0] - g[0]) + abs(p[1] - g[1])
79         rows, cols = linear_sum_assignment(cost)
80         matches: List[Match] = []
81         for i, j in zip(rows.tolist(), cols.tolist()):
82             if i < n and j < m and cost[i, j] < big: # drop forced ineligible assignments
83                 matches.append((i, j, preds[i][0] - gts[j][0], preds[i][1] - gts[j][1]))
84         matches.sort(key=lambda mm: mm[1])
85         return matches
86
87
88     def match_1to1(preds: List[Interval], gts: List[Interval],
89         tau_start: float = TAU_START, tau_end: float = TAU_END
90         ) -> Tuple[List[Match], float, float, float]:
91         """Strict 1:1 tolerance match ? ``(matches, precision, recall, f1)``. ``P = matched

```



```

/ n_pred``
92         (over-production costs precision), ``R = matched / n_gt``. Exact Hungarian when
scipy is
93         present; deterministic greedy otherwise."""
94         n, m = len(preds), len(gts)
95         if n == 0 or m == 0:
96             return [], 0.0, 0.0, 0.0
97         matches = _hungarian_match_1to1(preds, gts, tau_start, tau_end)
98         if matches is None:
99             matches = _greedy_match_1to1(preds, gts, tau_start, tau_end)
100         k = len(matches)
101         precision = k / n
102         recall = k / m
103         f1 = (2 * precision * recall / (precision + recall)) if (precision + recall) else
0.0
104         return matches, precision, recall, f1
105
106
107     def _percentile(xs: List[float], p: float) -> float:
108         """Linear-interpolated percentile (pure stdlib)."""
109         if not xs:
110             return 0.0
111         s = sorted(xs)
112         k = (len(s) - 1) * p / 100.0
113         lo = int(k)
114         hi = min(lo + 1, len(s) - 1)
115         return s[lo] + (s[hi] - s[lo]) * (k - lo)
116
117
118     def _overlap(a: Interval, b: Interval) -> float:
119         return max(0.0, min(a[1], b[1]) - max(a[0], b[0]))
120
121
122     def boundary_error_report(preds: List[Interval], gts: List[Interval]) -> Dict[str,
Dict[str, float]]:
123         """Median + IQR of signed AND absolute ?start, ?end, split start/end ? the "start
solved,
124         end open" diagnostic that aims R4.
125
126         Computed over **best-overlap** matches (each GT ? its maximally-overlapping
prediction, any
127         positive overlap), NOT the within-? 1:1 matches: a ?-gated set is survivorship-
biased (loose
128         ends fail the gate and vanish from the distribution), which would HIDE the very end-
deficit
129         this diagnostic exists to surface. Unconditional best-overlap is what the day-1
validation used
130         to find |?end| ? |?start|. (The within-? deficit shows up instead as low
``tol_recall``.)"""
131         def dist(vals: List[float]) -> Dict[str, float]:
132             av = [abs(v) for v in vals]
133             return {
134                 "signed_median": statistics.median(vals) if vals else 0.0,
135                 "abs_median": statistics.median(av) if av else 0.0,
136                 "abs_p25": _percentile(av, 25.0),
137                 "abs_p75": _percentile(av, 75.0),
138                 "n": float(len(vals)),
139             }
140         ds: List[float] = []
141         de: List[float] = []
142         for g in gts:
143             best = max(preds, key=lambda p: _overlap(p, g), default=None)
144             if best is not None and _overlap(best, g) > 0.0:
145                 ds.append(best[0] - g[0])
146                 de.append(best[1] - g[1])

```

```

147         return {"dstart": dist(ds), "dend": dist(de)}
148
149
150     def over_seg_report(preds: List[Interval], gts: List[Interval]) -> Dict[str, int]:
151         """The over-segmentation GUARD (per-video no-regression in promotion):
152         ``split_count`` (?2
153         preds covering one rally) + ``merge_count`` (one pred ? ?2 rallies). Reuses the
154         bipartite
155         overlap-graph in ``segmentation_metrics.merge_split_report``.
156         from backend.eval.segmentation_metrics import merge_split_report
157         r = merge_split_report(preds, gts)
158         return {"split_count": int(r["splits"]), "merge_count": int(r["merges"])}
159
160     # ----- #
161     # R1 ? net over-segmentation promotion guard
162     # ----- #
163     #: Default over-seg cost weights. A MERGE hides a rally (?2 golden rallies collapse into
164     one
165     #: predicted window ? the user can't reach the hidden one ? recall loss at rally
166     granularity); a
167     #: SPLIT only fragments one rally (it is still found, just in pieces). So a merge is the
168     strictly
169     #: worse fault and is weighted higher. (Equal weights also pass the milestone ? see the
170     R1 evidence;
171     #: the asymmetry is the principled default, not a number tuned to force a verdict.)
172     WEIGHT_MERGE: float = 2.0
173     WEIGHT_SPLIT: float = 1.0
174
175     def over_seg_guard(base: List[Dict[str, Any]], cand: List[Dict[str, Any]], *,
176                       weight_merge: float = WEIGHT_MERGE, weight_split: float =
177                       WEIGHT_SPLIT,
178                       use_rates: bool = True, eps: float = 1e-9) -> Dict[str, Any]:
179         """**Net** over-segmentation promotion guard (the R1 refinement of the strict per-
180         video rule).
181         ``base``/``cand`` are aligned per-video over-seg dicts (the rows ``rally_seg_eval``
182         already
183         emits): each needs ``split_count``/``merge_count`` and ? when ``use_rates``
184         (default) ?
185         ``merge_rate``/``split_rate``. Optional ``name`` keys align the two lists by video
186         (else by
187         position). Returns the promote-or-block verdict for promoting ``cand`` over
188         ``base``.
189
190         **Why the strict rule needed refining.** R2 §2.3.3's guard blocks promotion if
191         ``cand`` raises
192         ``split_count`` OR ``merge_count`` on ANY video. That is right for an *incremental*
193         cue (same
194         windowing structure, an increase = a real regression) but mis-fires on a
195         *structural* change
196         (mega-windows ? bounded): splitting a single mega-window necessarily lifts
197         ``split_count`` from
198         ~0, and the raw merge **count** is non-monotonic ? one mega-window swallowing 40
199         rallies is
200         ``merge_count=1`` yet ``merge_rate=1.0`` (every rally hidden), whereas bounding it
201         into pieces
202         that each glue 2 neighbours is ``merge_count=9`` yet ``merge_rate=0.5`` (FEWER
203         rallies hidden).
204         So the strict count rule blocks a config that strictly *reduces* the fraction of
205         rallies lost to
206         over-merge. (Measured: the cap6+R4 milestone trips strict on 10/11 videos yet cuts
207         mean
208         ``merge_rate`` 0.694 ? 0.150 with NO per-video merge_rate regression ?

```

```

RALLY_QUALITY_RESEARCH $6.)
191
192     **The net rule (default verdict ``passes``):** score over-seg as ONE weighted cost
per video and
193     require BOTH:
194         1. the corpus-mean net cost does not rise (``cand_net ? base_net + eps``), AND
195         2. NO video's **merge_rate** rises beyond ``eps`` ? reintroducing a mega-window
that hides MORE
196         of a video's rallies is the one over-merge regression that is never acceptable,
the honest
197         "no NEW merges" rule expressed on the *rate* (the recall-meaningful quantity),
not the count.
198         With ``use_rates`` the per-video cost uses ``merge_rate``/``split_rate`` (fraction
of GT rallies
199         affected, ? [0,1] ? comparable across a structural change); ``use_rates=False``
scores raw counts.
200
201     **Revert (one line):** read ``strict_passes`` instead of ``passes`` to fall back to
the original
202     strict per-video count rule. Both verdicts are always reported, so the choice is
auditable.
203     """
204     def _aligned() -> List[Tuple[Dict[str, Any], Dict[str, Any]]]:
205         if base and cand and all("name" in b for b in base) and all("name" in c for c in
cand):
206             cmap = {c["name"]: c for c in cand}
207             return [(b, cmap[b["name"]]) for b in base if b["name"] in cmap]
208             n = min(len(base), len(cand))
209             return [(base[i], cand[i]) for i in range(n)]
210
211     def _cost(row: Dict[str, Any]) -> float:
212         if use_rates:
213             return weight_merge * float(row["merge_rate"]) + weight_split *
float(row["split_rate"])
214             return weight_merge * float(row["merge_count"]) + weight_split *
float(row["split_count"])
215
216     pairs = _aligned()
217     n = len(pairs)
218     base_costs = [_cost(b) for b, _ in pairs]
219     cand_costs = [_cost(c) for _, c in pairs]
220     base_net = (sum(base_costs) / n) if n else 0.0
221     cand_net = (sum(cand_costs) / n) if n else 0.0
222     net_delta = cand_net - base_net
223
224     # Per-video regressions, reported for transparency.
225     new_merges: List[Dict[str, Any]] = []          # raw COUNT increases (what strict
trips on)
226     new_splits: List[Dict[str, Any]] = []          # raw COUNT increases (what strict
trips on)
227     merge_rate_regress: List[Dict[str, Any]] = []  # RATE increases (the real over-
merge regression)
228     for b, c in pairs:
229         if float(c["merge_count"]) > float(b["merge_count"]) + eps:
230             new_merges.append({"name": b.get("name", ""),
231                                "base": float(b["merge_count"]), "cand":
float(c["merge_count"])})
232         if float(c["split_count"]) > float(b["split_count"]) + eps:
233             new_splits.append({"name": b.get("name", ""),
234                                "base": float(b["split_count"]), "cand":
float(c["split_count"])})
235         if use_rates and float(c["merge_rate"]) > float(b["merge_rate"]) + eps:
236             merge_rate_regress.append({"name": b.get("name", ""),
237                                         "base": float(b["merge_rate"]), "cand":
float(c["merge_rate"])})

```

```

238
239     strict_passes = (not new_merges) and (not new_splits)
240     net_cost_ok = net_delta <= eps
241     no_merge_rate_regress = not merge_rate_regress
242     passes = bool(net_cost_ok and no_merge_rate_regress)
243     return {
244         "passes": passes,
245         "strict_passes": strict_passes,
246         "net_cost_ok": net_cost_ok,
247         "no_merge_rate_regress": no_merge_rate_regress,
248         "base_net": base_net,
249         "cand_net": cand_net,
250         "net_delta": net_delta,
251         "n": n,
252         "use_rates": use_rates,
253         "weight_merge": weight_merge,
254         "weight_split": weight_split,
255         "new_merges": new_merges,
256         "new_splits": new_splits,
257         "merge_rate_regress": merge_rate_regress,
258     }
259
260
261     def tolerance_report(preds: List[Interval], gts: List[Interval],
262                         tau_start: float = TAU_START, tau_end: float = TAU_END) ->
Dict[str, object]:
263         """The full R2 block for one video: P/R/F1@? + boundary-error distributions + over-
seg guard."""
264         matches, precision, recall, f1 = match_1to1(preds, gts, tau_start, tau_end)
265         return {
266             "tau_start": tau_start, "tau_end": tau_end,
267             "n_pred": len(preds), "n_gt": len(gts), "matched": len(matches),
268             "precision": precision, "recall": recall, "f1": f1,
269             "boundary_error": boundary_error_report(preds, gts),
270             "over_seg": over_seg_report(preds, gts),
271         }
272
273
274     def tolerance_sweep(preds: List[Interval], gts: List[Interval],
275                        taus: Tuple[float, ...] = SWEEP_TAUS) -> Dict[float, float]:
276         """Symmetric-? F1 sweep ? the trend curve (a single ? misleads). Monotonic non-
decreasing in ?."""
277         return {t: match_1to1(preds, gts, t, t)[3] for t in taus}
278

```

5. assistant (2026-06-19T15:52:12.398Z)

Now let me verify the cited `rally\_seg\_eval.py` to confirm the claims about CLI flags and the report format.

6. user (2026-06-19T15:52:13.846Z)

```

330         f"/md={p['inactivity_min_density']})"
331     if p.get("blob_fallback"):
332         bounded.append("blob_fallback")
333     extra = (" " + " ".join(bounded)) if bounded else ""
334     md = result.get("min_duration", 0.0) or 0.0
335     md_tag = f", LONG-RALLY LENS min_dur>{md:g}s" if md > 0 else ""
336     lines = [f"# Rally-segmentation eval ? windowing '{p['name']}' "
337             f"(vt={p['velocity_thresh']}, merge_gap={p['merge_gap']}, "
338             f"min_win={p['min_window_duration']}{extra}), IoU={result['iou']}{md_tag}",
339             ""]
340     lines.append("| video | F1 | F1@0.25 | F1@0.5 | merges | merge_rate | seg_ratio |
pred/gt |")
340     lines.append("|---|---|---|---|---|---|---|")

```

```

341         for r in result["rows"]:
342             lines.append(f"| {r['name']} | {r['f1']:.3f} | {r['f1@0.25']:.3f} |
343 {r['f1@0.5']:.3f} | "
344                             f"{int(r['merges'])} | {r['merge_rate']:.2f} |
345 {r['segment_count_ratio']:.2f} | "
346                             f"{r['num_pred']}/{r['num_gt']} |")
345         a = result["aggregate"]
346         lines += [f"***Aggregate (n={a['n']}, mean [95% CI]):** "
347                     f"F1 {_ci(a['f1'])} · F1@0.25 {_ci(a['f1@0.25'])} · F1@0.5
348 {_ci(a['f1@0.5'])} · "
349                     f"merge_rate {_ci(a['merge_rate'])} · seg_ratio
350 {_ci(a['segment_count_ratio'])}"]
349         # --- R2 (tolerance-match) block ? the task-faithful headline (tIoU above is
secondary) ---
350         rows = result["rows"]
351         if rows and "tol_f1" in rows[0]:
352             ts, te = rows[0]["tau_start"], rows[0]["tau_end"]
353             lines += [f"### R2 ? tolerance-match (?_start={ts}s, ?_end={te}s, strict
1:1)", "",
354                     "| video | tolF1 | tolP | tolR | |?start| | |?end| | split | merge |",
355                     "|---|---|---|---|---|---|---|---|"]
356         for r in rows:
357             lines.append(f"| {r['name']} | {r['tol_f1']:.3f} | {r['tol_precision']:.2f}
| "
358                             f"{r['tol_recall']:.2f} | {r['dstart_abs_median']:.2f} | "
359                             f"{r['dend_abs_median']:.2f} | {int(r['split_count'])} |
360 {int(r['merge_count'])} |")
360             lines += [f"***R2 aggregate (n={a['n']}):** tolF1 {_ci(a['tol_f1'])} · "
361                         f"tolP {_ci(a['tol_precision'])} · tolR {_ci(a['tol_recall'])} · "
362                         f"||?start| {_ci(a['dstart_abs_median'])} · |?end|
363 {_ci(a['dend_abs_median'])} "
364                         f"(start?solved / end=the gap)"]
364         return "\n".join(lines)
365
366
367     def format_stratified(strat: Dict[str, Any]) -> str:
368         """Render the long-rally stratified table: {all vs long} × {single, multicourt, all-
courts}.
369
370         ``tolP`` is the column that carries the story ? if filtering to long rallies strips
local's
371         short false positives, precision rises (most on multicourt). ``n`` is the videos
with >=1 golden
372         rally in that stratum."""
373         long_tau = strat["long_tau"]
374         cells = strat["cells"]
375         lines = [f"\n## Long-rally lens ? stratified (long = duration > {long_tau:g}s; "
376                 "n = videos with >=1 golden rally in the stratum)", "",
377                 "| stratum | court | n | tolF1 | tolP | tolR | tIoU F1@0.5 | seg_ratio |",
378                 "|---|---|---|---|---|---|---|---|"]
379         for dur_label in strat["labels"]:
380             for court in strat["courts"]:
381                 a = cells[(dur_label, court)]
382                 lines.append(f"| {dur_label} | {court} | {a['n']} | {_ci(a['tol_f1'])} | "
383                             f"{_ci(a['tol_precision'])} | {_ci(a['tol_recall'])} | "
384                             f"{_ci(a['f1@0.5'])} | {_ci(a['segment_count_ratio'])} |")
385         return "\n".join(lines)
386
387
388     def _resolve_preset(name: str) -> WindowingPreset:
389         if name in PRESETS:
390             return PRESETS[name]
391         raise SystemExit(f"unknown preset '{name}'; choices: {sorted(PRESETS)}")
392
393

```

```

394     def _build_preset(args: argparse.Namespace, side: str) -> WindowingPreset:
395         """Resolve the ``base`` (``--preset``) or ``cand`` (``--vs``) windowing from
396         ``args`` by
397         applying the ``dataclasses.replace`` overrides for cap / hard-cap / blob-fallback /
398         inactivity. ``cand`` falls back to the base override when its ``--vs-*`` variant is
399         unset
400         (so a baseline-vs-candidate promotion check runs in one shot)."""
401         from dataclasses import replace
402         if side == "base":
403             preset = _resolve_preset(args.preset)
404             cap = args.max_window_duration
405             hard_cap = args.hard_cap
406             blob_fallback = args.blob_fallback
407             inactivity_end = args.inactivity_end
408         else:
409             preset = _resolve_preset(args.vs)
410             cap = (args.vs_max_window_duration if args.vs_max_window_duration is not None
411                   else args.max_window_duration)
412             hard_cap = args.vs_hard_cap or args.hard_cap
413             blob_fallback = args.vs_blob_fallback or args.blob_fallback
414             inactivity_end = (args.vs_inactivity_end if args.vs_inactivity_end is not None
415                              else args.inactivity_end)
416         if cap is not None:
417             preset = replace(preset, max_window_duration=cap)
418         if hard_cap:
419             preset = replace(preset, hard_cap=True)
420         if blob_fallback:
421             preset = replace(preset, blob_fallback=True)
422         if args.blob_factor is not None:
423             preset = replace(preset, blob_factor=args.blob_factor)
424         if inactivity_end is not None:
425             preset = replace(preset, inactivity_end=inactivity_end)
426         if args.inactivity_rally_thresh is not None:
427             preset = replace(preset, inactivity_rally_thresh=args.inactivity_rally_thresh)
428         if args.inactivity_min_density is not None:
429             preset = replace(preset, inactivity_min_density=args.inactivity_min_density)
430         return preset
431
432     def main() -> None:
433         ap = argparse.ArgumentParser(description="Rally-segmentation eval vs golden labels
434         (offline).")
435         ap.add_argument("--manifest", default=DEFAULT_MANIFEST)
436         ap.add_argument("--features-dir", default=DEFAULT_FEATURES_DIR)
437         ap.add_argument("--preset", default=SERVED.name, help="windowing preset (default:
438         served)")
439         ap.add_argument("--vs", default=None, help="second preset to diff against --preset")
440         ap.add_argument("--iou", type=float, default=DEFAULT_IOU)
441         ap.add_argument("--min-crossings", type=int, default=0,
442             help="net-crossings rally-state gate on --preset (0=off; W2
443             Gate-A)")
444         ap.add_argument("--vs-min-crossings", type=int, default=0,
445             help="net-crossings gate on --vs (default: same as --min-
446             crossings)")
447         ap.add_argument("--max-window-duration", type=float, default=None,
448             help="override bounded-windowing cap (s) on BOTH presets (W1; e.g.
449             12)")
450         ap.add_argument("--vs-max-window-duration", type=float, default=None,
451             help="cap (s) on --vs ONLY (default: same as --max-window-duration);
452             lets a "
453             "BASELINE (no cap) vs CANDIDATE (capped) promotion check run in
454             one shot")
455         ap.add_argument("--hard-cap", action="store_true",
456             help="enforce the cap as a HARD cap on --preset (chop continuously-
457             active "

```

```

449         "windows at the cap when no inactivity gap exists; W1 hard-cap
fallback)")
450     ap.add_argument("--vs-hard-cap", action="store_true",
451         help="hard-cap fallback on --vs (default: same as --hard-cap)")
452     ap.add_argument("--blob-fallback", action="store_true",
453         help="R5 blob-fallback on --preset (after R4, force-chop any group
still over the "
454         "cap at its midpoint + flag/log it; the continuously-active
blob last resort)")
455     ap.add_argument("--vs-blob-fallback", action="store_true",
456         help="R5 blob-fallback on --vs (default: same as --blob-fallback)")
457     ap.add_argument("--blob-factor", type=float, default=None,
458         help="R5 magnitude gate (both presets): chop only groups > this ×
cap (default 4.0)")
459     ap.add_argument("--inactivity-end", type=float, default=None,
460         help="R4 rally-end inactivity trim (s) on --preset (0=off; trims
post-rally drift tail)")
461     ap.add_argument("--vs-inactivity-end", type=float, default=None,
462         help="R4 inactivity trim on --vs (default: same as --inactivity-
end)")
463     ap.add_argument("--inactivity-rally-thresh", type=float, default=None,
464         help="R4 velocity above which motion counts as 'rally' (both
presets; default 0.08)")
465     ap.add_argument("--inactivity-min-density", type=float, default=None,
466         help="R4 min trailing fast-fraction to stay 'in rally' (both
presets; default 0.34)")
467     ap.add_argument("--tau-start", type=float, default=None,
468         help="R2 tolerance-match start window (s); default 2.0 (forgives the
service window)")
469     ap.add_argument("--tau-end", type=float, default=None,
470         help="R2 tolerance-match end window (s); default 1.5 (keeps the
rally-end honest)")
471     ap.add_argument("--guard-weight-merge", type=float, default=None,
472         help="R1 over-seg guard: merge cost weight (default 2.0; merge hides
a rally)")
473     ap.add_argument("--guard-weight-split", type=float, default=None,
474         help="R1 over-seg guard: split cost weight (default 1.0; split only
fragments)")
475     ap.add_argument("--guard-counts", action="store_true",
476         help="R1 over-seg guard scores raw counts instead of normalized
rates "
477         "(rates are the default ? comparable across a structural
mega?bounded change)")
478     ap.add_argument("--min-duration", type=float, default=0.0,
479         help="LONG-RALLY LENS (0=OFF): score ONLY rallies longer than this
many seconds, "
480         "filtering BOTH predictions and golden GTs before matching.
Strips local's "
481         "short false-positive blips to reveal long-rally quality.
Applies across the "
482         "tIoU + R2 paths and to --vs (same filter on both sides).")
483     ap.add_argument("--stratify", action="store_true",
484         help="also print the long-rally stratified report: {all vs
long(>--long-tau)} × "
485         "{single vs multicourt} aggregates (court type from
GOLDEN_VIDEOS.md / the "
486         "manifest court_type field). The actual long-rally payoff
table.")
487     ap.add_argument("--long-tau", type=float, default=LONG_RALLY_TAU,
488         help=f"the 'long' rally duration threshold (s) used by --stratify "
489         f"(default {LONG_RALLY_TAU:g}; owner's North-star happy-path is
>5s).")

```

No matches found

8. assistant (2026-06-19T15:52:18.230Z)

Let me verify the rest of `main()` to confirm there's no `--tau-sweep` flag, and check whether `tolerance\_sweep` is referenced anywhere in `rally\_seg\_eval.py`.

9. user (2026-06-19T15:52:19.671Z)

```
489             f"(default {LONG_RALLY_TAU:g}; owner's North-star happy-path is
>5s).")
490     ap.add_argument("--json-out", default=None, help="optional path to write the full
result JSON")
491     args = ap.parse_args()
492
493     golden = load_golden(args.manifest, args.features_dir)
494     base = _build_preset(args, "base")
495     if args.vs:
496         vs_mc = args.vs_min_crossings or args.min_crossings
497         cand = _build_preset(args, "cand")
498         result = compare(golden, base, cand, args.iou,
499                         base_min_crossings=args.min_crossings,
cand_min_crossings=vs_mc,
500                         tau_start=args.tau_start, tau_end=args.tau_end,
501                         guard_weight_merge=args.guard_weight_merge,
502                         guard_weight_split=args.guard_weight_split,
503                         guard_use_rates=not args.guard_counts,
504                         min_duration=args.min_duration)
505     print(format_report(result["base"]))
506     print("\n" + format_report(result["cand"]))
507     d = result["delta"]
508     print(f"\n## ? ({args.vs} mc={vs_mc} ? {args.preset} mc={args.min_crossings}), "
509           f"n_paired={result['n_paired']}")
510     for k, dd in d.items():
511         st = dd["sign_test"]
512         print(f"  ?{k}: {dd['mean_delta']:+.3f}
[{dd['ci_low']:+.3f}, {dd['ci_high']:+.3f}] "
513               f"CI?0={dd['ci_excludes_zero']} sign(W/L)={st['wins']}/{st['losses']}
p={st['p_value']:.3f}")
514     print(_format_guard(result["over_seg_guard"]))
515     else:
516         result = evaluate(golden, base, args.iou, args.min_crossings,
517                           tau_start=args.tau_start, tau_end=args.tau_end,
518                           min_duration=args.min_duration)
519     print(format_report(result))
520     if args.stratify:
521         # The long-rally payoff table always reports the BASE preset (the served/current
config),
522         # split by duration x court type. Independent of --min-duration (it does its own
split).
523         print(format_stratified(stratified_report(
524             golden, base, args.iou, args.min_crossings,
525             tau_start=args.tau_start, tau_end=args.tau_end, long_tau=args.long_tau)))
526     if args.json_out:
527         with open(args.json_out, "w", encoding="utf-8") as fh:
528             json.dump(result, fh, indent=2)
529         print(f"\n[written {args.json_out}]")
530
531
532     if __name__ == "__main__":
533         main()
534
```

10. user (2026-06-19T15:52:21.037Z)


```

89:     Also carries the **R2** task-faithful block (`tolerance_metrics`, the headline going
forward;
98:     from backend.eval import tolerance_metrics as tm
107:     be = tm.boundary_error_report(preds, gts) # unconditional best-overlap (the R4-aiming
diagnostic)
250:     from backend.eval import tolerance_metrics as tm

```

11. assistant (2026-06-19T15:52:24.826Z)

Let me check lines 89-110 and around 250 of `rally\_seg\_eval.py` to confirm `tolerance\_sweep` is genuinely never called, and verify the report does not render a sweep table.

12. user (2026-06-19T15:52:26.274Z)

```

85         Pairs temporal-IoU P/R/F1 + boundary error (`metrics.score`) with the
86         over-segmentation-sensitive set (F1@k, segment-count ratio, merge/split breakdown)
so a
87         merged mega-window is *visible* even when IoU-F1 isn't moved.
88
89         Also carries the **R2** task-faithful block (`tolerance_metrics`, the headline
going forward;
90         tIoU here is the SECONDARY trend-line per the augment-then-ablation-gate decision):
strict-1:1
91         tolerance-match `tol_f1/tol_precision/tol_recall` (over-production costs
precision) + the
92         start/end boundary-error medians + the over-seg guard counts. See
docs/R2_EVAL_METRIC_DESIGN.md.
93
94         ``min_duration`` > 0 applies the **long-rally lens**
(:func:`filter_by_min_duration`): BOTH preds
95         and GTs are filtered to rallies longer than that many seconds BEFORE any matching,
so the whole
96         row (tIoU set AND R2 block) scores only long rallies. Default 0.0 = OFF (exact
passthrough).
97         """
98         from backend.eval import tolerance_metrics as tm
99         preds = filter_by_min_duration(preds, min_duration)
100         gts = filter_by_min_duration(gts, min_duration)
101         ts = tm.TAU_START if tau_start is None else tau_start
102         te = tm.TAU_END if tau_end is None else tau_end
103         sc = metrics.score(preds, gts, iou_threshold=iou)
104         f1k = segmentation_metrics.f1_at_overlaps(preds, gts)
105         ms = segmentation_metrics.merge_split_report(preds, gts)
106         tol_matches, tol_p, tol_r, tol_f1 = tm.match_1to1(preds, gts, ts, te)
107         be = tm.boundary_error_report(preds, gts) # unconditional best-overlap (the
R4-aiming diagnostic)
108         osr = tm.over_seg_report(preds, gts)
109         return {
110             "f1": sc.f1, "precision": sc.precision, "recall": sc.recall, "mean_iou":
sc.mean_iou,
111             "mean_start_error": sc.mean_start_error, "mean_end_error": sc.mean_end_error,
112             "f1@0.1": f1k[0.1], "f1@0.25": f1k[0.25], "f1@0.5": f1k[0.5],
113             "segment_count_ratio": segmentation_metrics.segment_count_ratio(preds, gts),
114             "merge_rate": ms["merge_rate"], "split_rate": ms["split_rate"],
115             "merges": ms["merges"], "splits": ms["splits"], "missed": ms["missed"],
116             "spurious": ms["spurious"], "num_pred": len(preds), "num_gt": len(gts),
117             # --- R2 (tolerance-match) ? the task-faithful headline block ---
118             "tau_start": ts, "tau_end": te,
119             "tol_f1": tol_f1, "tol_precision": tol_p, "tol_recall": tol_r,
120             "tol_matched": float(len(tol_matches)),
121             "dstart_abs_median": be["dstart"]["abs_median"], "dend_abs_median":
be["dend"]["abs_median"],
122             "split_count": osr["split_count"], "merge_count": osr["merge_count"],
123         }
124

```

13. user (2026-06-19T15:52:27.116Z)

```
245         over-segmentation metrics AND the **R1 net over-seg promotion guard**
(``over_seg_guard``):
246         the honest promote/block verdict for cand-over-base, with the strict per-video rule
reported
247         alongside so the refinement is auditable. ``min_duration`` > 0 scores both sides
through the
248         long-rally lens (same filter applied to base and cand ? it's a scoring lens, not a
windowing
249         knob, so it is identical on both sides of the A/B)."""
250         from backend.eval import tolerance_metrics as tm
251         b = evaluate(golden, base, iou, base_min_crossings, tau_start, tau_end,
min_duration)
252         c = evaluate(golden, cand, iou, cand_min_crossings, tau_start, tau_end,
min_duration)
253         by_name_b = {r["name"]: r for r in b["rows"]}
254         paired = [(by_name_b[r["name"]], r) for r in c["rows"] if r["name"] in by_name_b]
255         delta: Dict[str, Any] = {}
256         for k in ("f1", "f1@0.25", "tol_f1", "merge_rate", "split_rate",
"segment_count_ratio"):
257             delta[k] = eval_stats.paired_delta_ci([rb[k] for rb, _ in paired],
258                                                    [rc[k] for _, rc in paired])
259         gkw: Dict[str, Any] = {"use_rates": guard_use_rates}
260         if guard_weight_merge is not None:
261             gkw["weight_merge"] = guard_weight_merge
262         if guard_weight_split is not None:
263             gkw["weight_split"] = guard_weight_split
264         guard = tm.over_seg_guard([rb for rb, _ in paired], [rc for _, rc in paired], **gkw)
```

14. assistant (2026-06-19T15:52:39.254Z)

I have all the evidence needed. Let me verify the three exact `stale\_text` strings appear verbatim in the doc.

All three proposed fixes target text that ? per the current file I read ? is **already** in the corrected form¹. Let me confirm each carefully against the file lines.

Fix 1 ? proposed `stale\_text`:

```
...
- **Wire into `backend/eval/rally_seg_eval.py` ? emit the R2 block alongside tIoU (don't
remove tIoU);
  add `--tau-start/--tau-end/--tau-sweep`. Surface in the experiment leaderboard.
...
```

But the actual file (lines 147-149) reads:

```
...
- **Wire into `backend/eval/rally_seg_eval.py` ? emit the R2 block alongside tIoU (don't
remove tIoU);
  add `--tau-start/--tau-end` (both landed; the ?-sweep helper
`tolerance_metrics.tolerance_sweep` exists
  but is not yet surfaced as a `rally_seg_eval` CLI flag). Surface in the experiment
leaderboard.
...
```

The file is **already** the corrected text. The `stale\_text` does NOT appear verbatim. **DROP.**

Fix 2 ? proposed `stale\_text`:

```
...
- `tolerance_metrics.py` + tests merged (default-additive); `rally_seg_eval` reports the R2
decomposition +
  ?-sweep next to tIoU; the corpus (n?8) re-scored under R2 and recorded in QUALITY_ITERATIONS.
...
```

But the actual file (lines 166-168) reads:

```
...
- `tolerance_metrics.py` + tests merged (default-additive); `rally_seg_eval` reports the R2
decomposition
```

next to tIoU (the ?-sweep helper exists in `tolerance\_metrics` but is not yet wired into the `rally\_seg\_eval` report); the corpus (n?8) re-scored under R2 and recorded in QUALITY_ITERATIONS.

Already corrected. `stale\_text` does NOT appear verbatim. ****DROP.****

****Fix 3**** ? proposed `stale\_text`:

- `boundary\_error\_report(matches) -> {?start, ?end: signed/abs median+IQR}`.

But the actual file (lines 143-144) reads:

- `boundary\_error\_report(preds, gts) -> {?start, ?end: signed/abs median+IQR}` (computed over unconditional best-overlap matches internally ? see the refinement note above).

Already corrected. `stale\_text` does NOT appear verbatim. ****DROP.****

Let me confirm there is no other occurrence of these strings elsewhere in the file (e.g., the proposed stale signature, or a `--tau-sweep` mention).

15. user (2026-06-19T15:52:41.168Z)

178:****Locked (recommended, design-level):**** Hungarian 1:1 (not greedy/overlap); report the decomposition + sweep,

16. assistant (2026-06-19T15:52:48.351Z)

Confirmed. None of the three proposed `stale\_text` strings appear in the file. The file at `docs/R2\_EVAL\_METRIC\_DESIGN.md` has ****already been corrected**** to exactly the form each proposed fix wants (lines 147-149, 166-168, and 143-144 are already the "corrected_text"). The only `--tau-sweep`/sweep mention remaining is on line 178 ("report the decomposition + sweep"), which is a design-level statement about what the metric reports as a decomposition, not a claim about a wired CLI flag ? and it is not a target of any proposed fix.

Since the `stale\_text` for all three fixes cannot be matched verbatim in the current file (the staleness has already been fixed), all three must be dropped per the instructions.

17. user (2026-06-19T15:52:48.455Z)

Structured output provided successfully

18. assistant (2026-06-19T15:52:56.540Z)

All three proposed fixes were dropped. The file `docs/R2\_EVAL\_METRIC\_DESIGN.md` has already been corrected to exactly the form each fix proposed:

- ****Fix 1**** (`--tau-sweep` flag): The file at lines 147-149 already reads `add --tau-start/--tau-end` (both landed; the ?-sweep helper `tolerance\_metrics.tolerance\_sweep` exists but is not yet surfaced as a `rally\_seg\_eval` CLI flag). The proposed `stale\_text` (`--tau-start/--tau-end/--tau-sweep`) is not present.

- ****Fix 2**** (definition-of-done ?-sweep claim): Lines 166-168 already read the decomposition `next to tIoU (the ?-sweep helper exists in `tolerance_metrics` but is not yet wired...)`. The proposed `stale\_text` (`decomposition + ?-sweep next to tIoU`) is not present.

- ****Fix 3**** (`boundary\_error\_report` signature): Lines 143-144 already read `boundary\_error\_report(preds, gts) -> ...`. The proposed `stale\_text` (`boundary\_error\_report(matches)`) is not present.

The underlying code claims are accurate (verified: `rally\_seg\_eval.py` `main()` defines only `--tau-start`/`--tau-end` with no `--tau-sweep`; never imports/calls `tolerance\_metrics.tolerance\_sweep`; `boundary\_error\_report` is defined as `def boundary\_error\_report(preds, gts)` at `tolerance\_metrics.py:122`), but the document already reflects all three corrections, so there is nothing stale left to fix.